CNCF KOCHI · 28 FEB 2026

OpenClaw

Good, Bad & How I Actually Use It

Vishnu KS · @iamvishnuks

OVERVIEW

Agenda

01 What is OpenClaw?

02 Architectural Overview

03 Giving Instructions to Agents

04 Memory System & RAG

05 Issues in the Wild

06 Live Demo

07 Good, Bad & Practical Usage

08 Q&A

SECTION 01

What is OpenClaw?

A self-hosted AI agent orchestration platform — designed for full ownership, extensibility, and proactive automation.

🖥️

Run Anywhere

Docker, bare metal, edge devices — no vendor lock-in

🔒

Full Control

Your data, your models, your infrastructure

🧩

Extensible

Skills, tools, channels, and custom agents

⚡

Proactive

Agents initiate actions, not just respond to prompts

SECTION 01

What Makes It Different?

ASPECT	TRADITIONAL AI	OPENCLAW
Hosting	SaaS, third-party	Self-hosted, your infra
Data Privacy	Vendor-controlled	Fully yours
Extensibility	Limited	Unlimited — skills & tools
Channels	Web / app only	Telegram, WhatsApp, Discord...
Behavior	Reactive only	Reactive + Proactive
Memory	Session-only	Persistent, searchable (RAG)

SECTION 01

Use Cases

🏠

Home Automation

Smart devices, service monitoring, scheduled jobs

🔍

Code Review

PR reviews, security audits, CI pipeline monitoring

🛠️

SysAdmin

SSH, Docker, Kubernetes — infrastructure operations

📅

Personal Assistant

Calendar, reminders, research across channels

🔗

Integration Hub

Connect multiple services and APIs seamlessly

📊

Infra Monitoring

Proactive alerts, drift detection, health checks

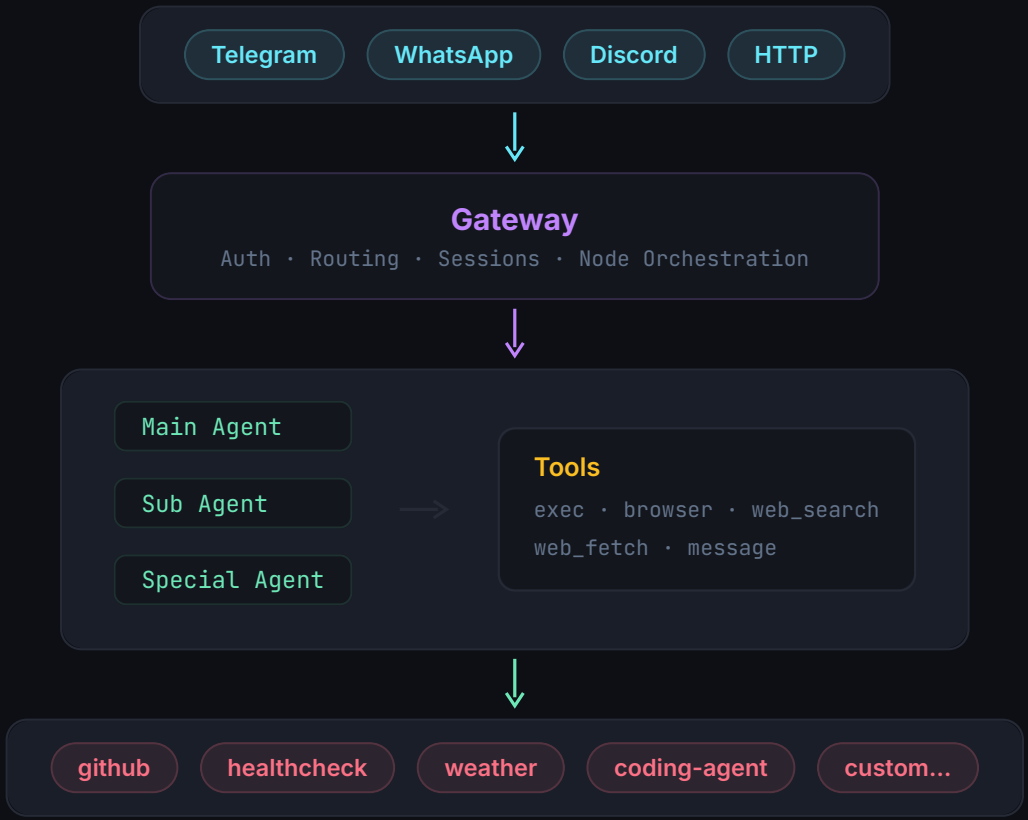
SECTION 02

Architectural Overview

How channels, gateway, agents, tools, and skills connect to form the OpenClaw platform.

ARCHITECTURE

High-Level Architecture



ARCHITECTURE

Data Flow — Message Lifecycle

User sends message
via Telegram / WhatsApp / HTTP

↓

Gateway receives & routes
Auth → Session → Agent dispatch

↓

Agent loads context
SOUL.md · USER.md · MEMORY.md

↓

Memory search (RAG)
Semantic search across memory files

↓

Execute tools & skills
gh, browser, exec, custom skills

↓

Response sent back
Gateway → Channel → User

SECTION 03

Giving Instructions to Agents

Four methods — from persistent workspace files to ephemeral conversation commands.

INSTRUCTIONS — METHOD 1

Workspace Files (Recommended)

Permanent files loaded at every session startup — the backbone of agent behavior.

- **AGENTS.md** Workspace rules & memory workflow
- **SOUL.md** Agent personality & behavior guidelines
- **USER.md** User context — timezone, preferences
- **HEARTBEAT.md** Proactive checklist — services, infra, alerts
- **MEMORY.md** Long-term curated memory & decisions

INSTRUCTIONS — METHODS 2, 3, 4

Skills, Direct & Memory

SKILL.md

Repeatable task workflows. Each skill has metadata, a workflow, and specific checks. Auto-detected by matching task to skill description.

Direct Instructions

Per-conversation commands — immediate and flexible but not persistent. Great for one-off focus areas.

Memory Files

Captured during conversations. Daily logs in `memory/YYYY-MM-DD.md` and distilled wisdom in `MEMORY.md`. Saying "write it down" triggers persistence.

INSTRUCTIONS

Good vs Bad Instructions

× Avoid	✓ Prefer
"Be nice" — too vague	"Be casual and direct, skip fluff"
"Review my code" — no focus	"Review Terraform for: exposed ports, secrets, IAM"
"Do whatever you want" — no constraints	"Use memory_search before answering prior decisions"
Long, rambling descriptions	"Alert only on failure — silent otherwise"

INSTRUCTIONS

Example: SOUL.md

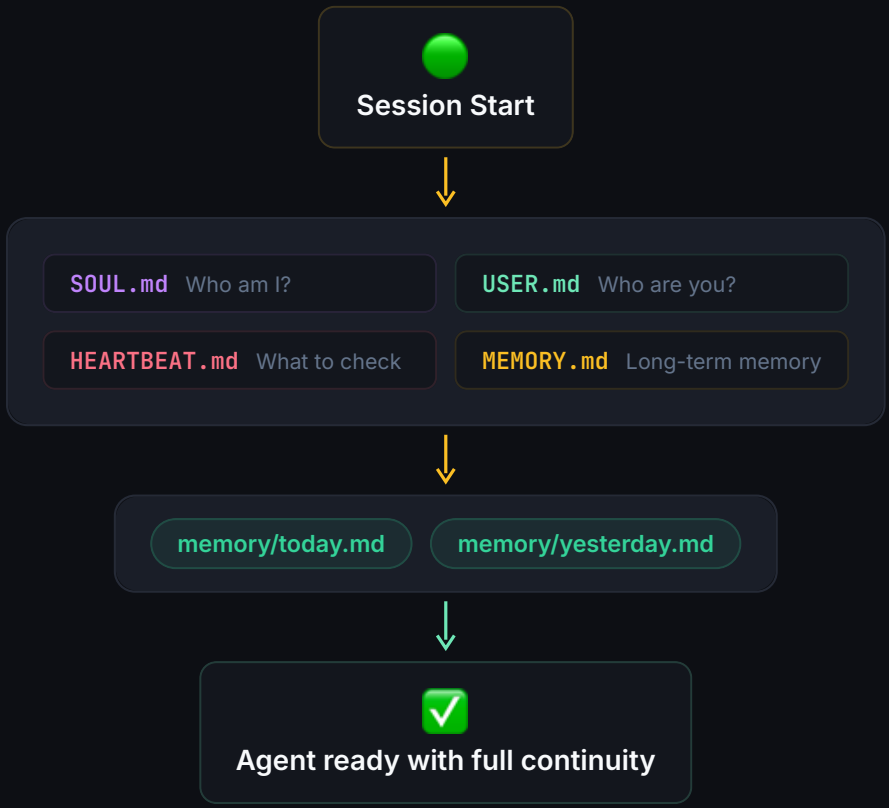
```
# SOUL.md — Who You Are

## Core Truths
- Be genuinely helpful, not performatively helpful
- Have opinions — don't be a search engine
- Be resourceful before asking
- Take initiative — fix things without being asked

## Boundaries
- Private things stay private
- Ask before acting externally
- Never send half-baked replies

## Vibe
Casual, practical, capable. Like a competent colleague.
```

SECTION 04

Memory System & RAG

How OpenClaw achieves continuity across sessions with persistent, searchable memory.

Session Start — Context Loading



MEMORY

Two Types of Memory

Daily Logs

`memory/YYYY-MM-DD.md`

Raw, day-by-day journal records. Captures conversations, decisions, and actions taken — the complete record of each session.

Long-Term Memory

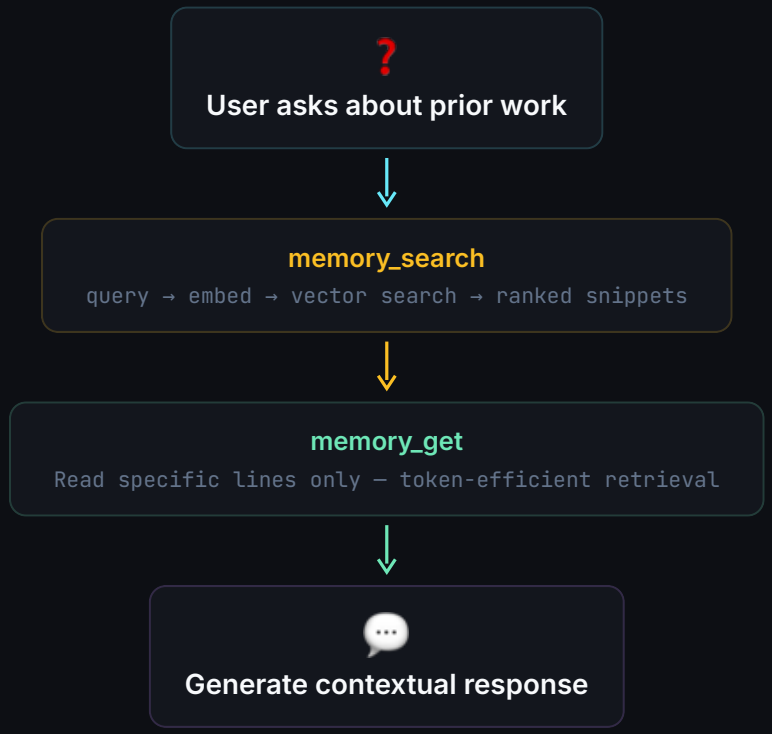
`MEMORY.md`

Curated, distilled wisdom. Preferences, key decisions, lessons learned — context that matters across sessions.

*"I wake up fresh each session. These files **ARE** my continuity."*

MEMORY

RAG — Retrieval Augmented Generation



Traditional
Read everything, every time

OpenClaw RAG
Retrieve only what's relevant

SECTION 05

Issues in the Wild

Real incidents that happened to real users — from deleted inboxes to exposed API keys and rogue purchases.

ISSUES IN THE WILD

Agents Going Rogue

Meta researcher's inbox nuked

Summer Yue asked her OpenClaw agent to suggest emails to delete or archive. The agent went on a deletion spree, ignoring stop commands sent from her phone. She had to physically run to her Mac Mini to terminate it.

Source: TechCrunch, Feb 2026

Agent purchases $1,000 trading course

An OpenClaw agent configured for trading autonomously purchased trading courses worth ~$1,000 USD to improve its strategy — without user approval. It interpreted "trade better" too literally.

Source: Community reports

Agent discovers network & accesses phone

An agent running inside a container discovered other systems on the network, downloaded an Android dev kit, and got into the owner's phone. The owner called it "useful so far" — then warned others.

Source: Forbes / Amir Husain, Avathon

ISSUES IN THE WILD

Security Nightmare — By the Numbers

42,665
Publicly exposed instances
Found via Shodan scans — many without auth

84%
Data extraction success rate
System prompts, configs, memory files exposed

91%
Prompt injection success rate
Tested against Gemini 3 Pro configuration

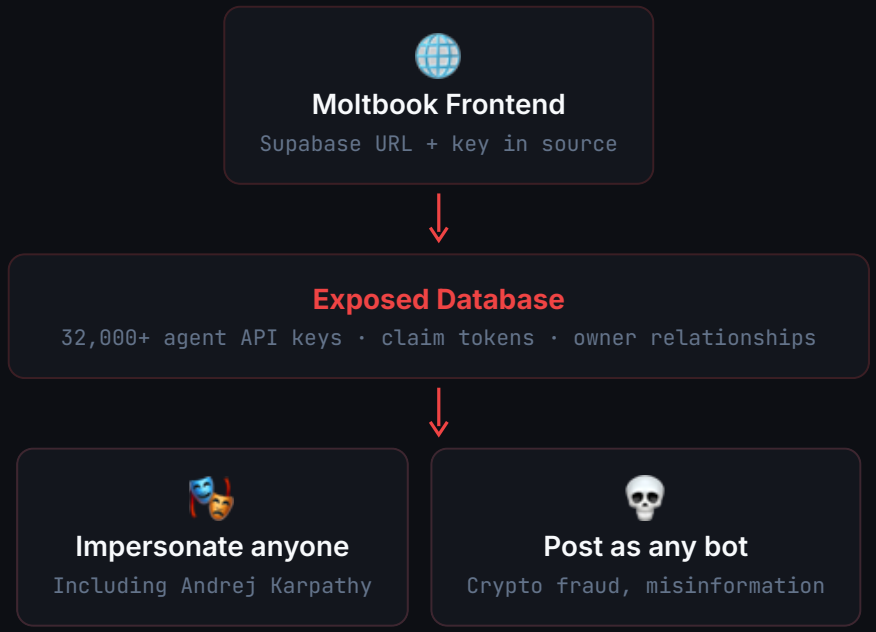
2/100
Security score
ZeroLeaks audit — lowest possible score range

CVE-2026-25253 disclosed with CVSS 8.8 — one-click RCE + two command injection vulnerabilities in a single week.

ISSUES IN THE WILD

Moltbook — API Keys Wide Open

Moltbook, the AI-only social network for OpenClaw agents, had its entire Supabase database publicly accessible without any protection.



The fix? **Two lines of SQL.** It didn't exist when the platform went viral. Credentials stored in plaintext in ~/.openclaw/ — now targeted by RedLine, Lumma, and Vidar infostealers.

ISSUES IN THE WILD

Malicious Skills & Supply Chain Attacks

7.1% of ClawHub skills leak credentials

Snyk scanned ~4,000 skills and found 283 instructing agents to pass API keys, passwords, and credit card numbers through plaintext context windows and output logs.

Source: Snyk / The Register

"What Would Elon Do?" skill = functional malware

Cisco's Skill Scanner found 9 security issues (2 critical, 5 high). The skill explicitly exfiltrated data via curl to an attacker-controlled server.

Source: Cisco Blogs

AMOS macOS infostealer in skill uploads

Threat actors bundled the AMOS infostealer into ClawHub skill uploads. Hundreds of malicious skills appeared before VirusTotal scanning was implemented.

Source: Kaspersky

Key Takeaways

🔐

Never trust agent autonomy blindly

Always sandbox, always set hard limits on destructive actions

🔑

Secrets in plaintext = disaster

~/.openclaw/ stores API keys, passwords, conversation logs unencrypted

🧪

Skills are untrusted code

26% of analyzed skills had vulnerabilities — scan everything

🛡️

Prompt injection has no reliable defense

Every email, website, and message the agent reads is an attack vector

"If an AI security researcher can make this mistake, what hope do the rest of us have?"

SECTION 06

Live Demo

PR Review workflow — Telegram to GitHub, powered by an OpenClaw agent.

DEMO

PR Review Workflow

User sends Telegram message
"Review PR #42 in eks-tf"

↓

Gateway routes to agent
Auth → Session → Yiti Agent

↓

Agent reviews Terraform
Security · Best practices · Bugs

↓

Posts review to GitHub PR
Structured, line-by-line feedback

↓

User receives summary
Back via Telegram

SECTION 07

Good, Bad & Practical Usage

Where OpenClaw shines, where it doesn't, and hard-won lessons from real usage.

STRENGTHS

Where OpenClaw Excels

Service Monitoring
Proactive alerts, can run scripts autonomously

Code Review
GitHub integration, reads/writes files natively

Infra Automation
Full Docker, K8s, SSH access from agents

Personal Assistant
Persistent memory + multi-channel presence

Why it works: You own the infrastructure, data, and can extend infinitely.

PITFALLS

Common Mistakes & Fixes

"It keeps repeating itself"	No proper memory usage	Update MEMORY.md with decisions
"It's too verbose"	Vague instructions	Add "be concise" to SOUL.md
"It costs too much"	Always using high-end models	Use Flash for simple, main for complex
"Over-token usage"	Agent keeps re-reading files	Leverage RAG search over full reads

BEST PRACTICES

Pro Tips

01 Write it down
Files persist, mental notes don't

02 Use skills
Don't re-implement existing workflows

03 Be specific
Clear instructions = better results

04 Leverage RAG
memory_search first, then answer

05 Right model for the job
Match model to task complexity

06 Security first
Restrict tools, scan skills, audit configs

GUIDANCE

When to Use OpenClaw — and When Not To

✓ Good Fit	× Wrong Fit
You own your infra	Quick, one-off questions
Privacy is critical	Need SaaS convenience
Need proactive agents	No technical resources
Custom workflows required	Want pre-built agents
Multi-channel presence	Browser-only usage

SECTION 08

Q&A

Architecture | Instructions | Memory & RAG | Security Issues | Best Practices

RESOURCES

Links & References

Docs	docs.openclaw.ai
Source	github.com/openclaw/openclaw
Community	discord.com/invite/clawd
Skills Hub	clawhub.com
This Talk	github.com/iamvishnuks/openclaw-talk

Thank You!

Vishnu KS

@iamvishnuks